Chapter 1

EXPERIMENTAL RESULTS AND ANALYSIS

This chapter presents the results of the full experimental evaluation and evaluates the central hypothesis: whether probability-based pruning can deliver meaningful speedup without sacrificing path optimality. Section 1.1 covers the 2,160-row synthetic benchmark; Section 1.2 covers the 840-row real-data validation on RetailRocket and RecSys 2015 (300 fixed-τ + 540 adaptive-τ).

1.1 Synthetic Experiment Results

Reading note. Throughout this chapter, "all-rows" speedups include runs where the pruned algorithm found no path (fast infeasibility detection), while "found-only" speedups restrict to runs that returned a path. Section 1.1.6 provides a detailed interpretation.

1.1.1 Speedup vs. Pruning Threshold τ

Table 1.1 summarizes median speedups aggregated across all graph types, sizes, degrees, and distributions.

τ	Edge Speedup	Exam. Speedup	Wall Speedup (all)	Wall Speedup (found)	Base (ms)	Pruned (ms)	Found (%)
0.001	8.7×	1.9×	3.3×	1.6×	4.98	1.561	30.6
0.01	79.2×	14.0×	24.7×	4.4×	4.98	0.206	5.6
0.05	368.4×	70.4×	123.6×	5.7×	4.98	0.040	4.7
0.1	810.5×	136.9×	242.6×	6.4×	4.98	0.020	4.4
0.5	1,986×	711.1×	738.2×	7.2×	4.98	0.006	4.2

Table 1.1: Median speedups by pruning threshold τ. "Wall Speedup (all)" includes rows where the pruned algorithm terminated without finding a path; "Wall Speedup (found)" is restricted to configurations where a path was returned.

All reported speedup values represent the median of per-configuration ratios (baseline time divided by pruned time), not the ratio of the per-column median times. Two wall-clock columns are provided: *Wall Speedup (all)* includes every configuration—including those where the pruned algorithm terminated without finding a path—while *Wall Speedup (found)* is restricted to configurations where a path was actually returned.

The all-rows speedups scale monotonically with τ, ranging from 3.3× at $\tau = 0.001$ to 738× at $\tau = 0.5$, but these figures are dominated by the ∼90% of rows in which the pruned algorithm quickly terminated without recovering a path. When restricted to path-found configurations only, the wall-clock speedup is markedly more modest: 1.6× at $\tau = 0.001$ rising to 7.2× at $\tau = 0.5$. This gap reflects a fundamental property of threshold-based pruning on Dijkstra's algorithm: when the optimal path survives the threshold, the pruned algorithm must still examine essentially the same set of edges because Dijkstra's priority-queue ordering already processes nodes in non-decreasing cost order. The modest found-only speedup arises from fewer heap-push operations on pruned edges, not from exploring a smaller portion of the graph.

The "Edge Speedup" column—based on `edges_relaxed`—consistently exceeds both other measures because its pruned denominator excludes threshold-pruned edges; it quantifies pruning aggressiveness, not raw work reduction. The "Exam. Speedup" column provides the fair like-for-like comparison using `edges_examined`, which counts every outgoing edge the algorithm inspects regardless of whether it survives the threshold (see Table **??** for counter semantics). Across all 178 path-found rows among the 1,800 pruned synthetic runs ($\tau > 0$), the optimality gap remained $\Delta = 0.0000\%$, confirming the all-or-nothing property (Theorem **??**, Chapter **??**). Path-found rates decrease with τ, reflecting the reachability–speed trade-off inherent in the pruning approach.

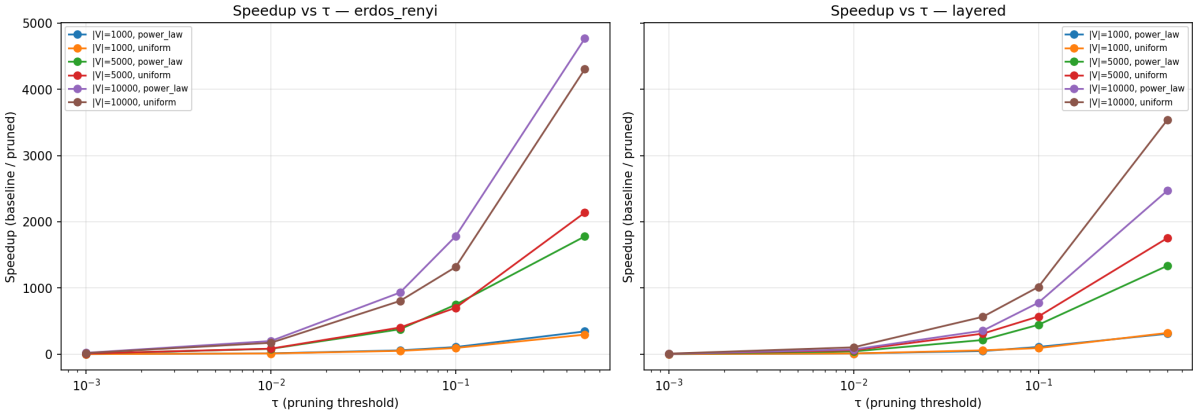

Figure 1.1: Wall-clock speedup vs. τ for ER (left) and layered (right) graphs.

Figure 1.1 plots the wall-clock speedup ratio (baseline time divided by pruned time) against the pruning threshold τ on a logarithmic x-axis spanning $\tau \in [0.001, 0.5]$. Each line represents a unique combination of graph size $|V|$ and edge-weight distribution. Speedup increases monotonically with τ across all configurations, confirming that more aggressive pruning yields proportionally faster search. Erdős–Rényi graphs consistently achieve higher speedups than layered graphs because their uniformly random connectivity exposes a larger fraction of low-probability edges to the threshold. In contrast, the layered generator's built-in forward bias toward the target means that many edges carry relatively high probability and therefore survive pruning. Larger graphs (higher $|V|$) produce steeper speedup curves, reflecting the greater absolute fraction of the search space that the threshold eliminates as the graph grows.

1.1.2 Speedup vs. Graph Size $|V|$

Table 1.2 shows how speedup scales with graph size at fixed $\tau = 0.01$.

$\|V\|$	Edge Spd	Exam. Spd	Wall Spd (all)	Wall Spd (found)	Base (ms)	Pruned (ms)	Peak Mem (KB)
1,000	31.5×	5.7×	9.4×	2.7×	1.67	0.189	83.8 → 4.4
5,000	104.0×	22.9×	43.1×	5.6×	7.12	0.215	342.9 → 4.5
10,000	180.4×	41.3×	77.0×	6.3×	15.83	0.222	655.0 → 4.5

Table 1.2: Median speedups at $\tau = 0.01$ by graph size. "All" includes termination-without-path rows; "found" is restricted to path-found configurations.

Baseline execution time scales linearly with $|V|$ (1.67 → 7.12 → 15.83 ms for $|V| =$ 1,000 → 5,000 → 10,000), consistent with the $O(|V| \log |V|)$ theoretical bound for sparse graphs. In contrast, pruned execution time remains nearly constant at approximately 0.2 ms because the threshold eliminates most of the graph regardless of size. All-rows speedup therefore grows with $|V|$: from 9.4× at $|V| = 1{,}000$ to 77× at $|V| = 10{,}000$. Found-only speedups are more modest (2.7× to 6.3×) but nonetheless increase with graph size, indicating that pruning saves more heap-push work on larger graphs. Peak memory for the pruned algorithm is essentially constant at approximately 4.5 KB, whereas baseline memory grows proportionally to $|V|$.

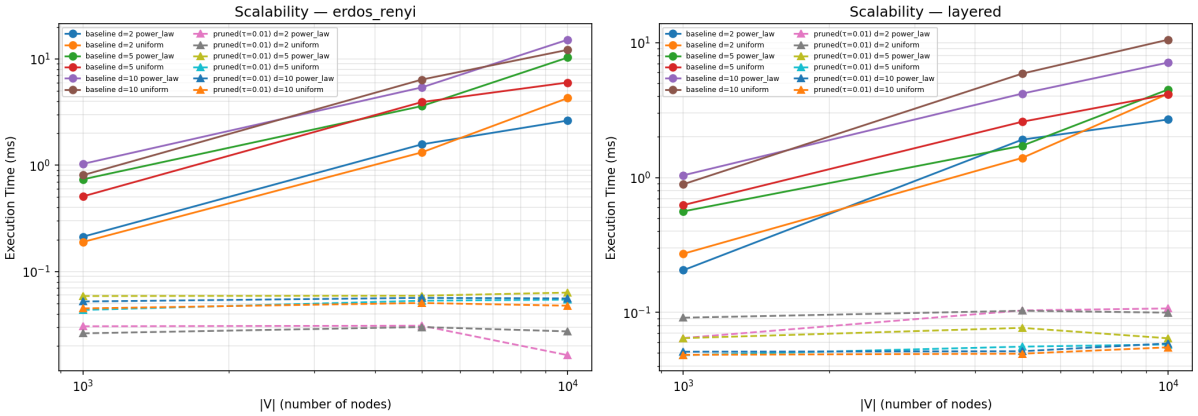

Figure 1.2: Execution time (ms) vs. $|V|$ at $\tau = 0.01$ on log–log axes for baseline and pruned Dijkstra across ER (left) and layered (right) graphs. Solid lines represent baseline; dashed lines represent the pruned variant.

Figure 1.2 visualises this scaling behaviour on log–log axes. Baseline execution time (solid lines) rises linearly with $|V|$, whereas pruned execution time (dashed lines) remains nearly flat across graph sizes—clustered around 0.03–0.06 ms for ER graphs and 0.03–0.10 ms for layered graphs. (The table medians of ∼0.2 ms are higher because they aggregate across all graph types, degrees, and distributions at $\tau = 0.01$; the figure shows individual-configuration lines.) The widening gap between the two sets of curves confirms that the pruning advantage grows with $|V|$, consistent with the theoretical $O(|V| \log |V|)$ baseline versus effectively constant pruned complexity established in Chapter **??**. Higher average degree d shifts both baseline and pruned curves upward, but the relative separation is preserved.

Figure 1.3 displays peak memory consumption on log–log axes for Erdős–Rényi (left) and layered (right) graphs at $\tau = 0.01$. Solid lines with circle markers represent baseline Dijkstra; dashed lines with triangle markers represent the pruned variant. Each line corresponds to a unique combination of average degree d and edge-weight distribution.

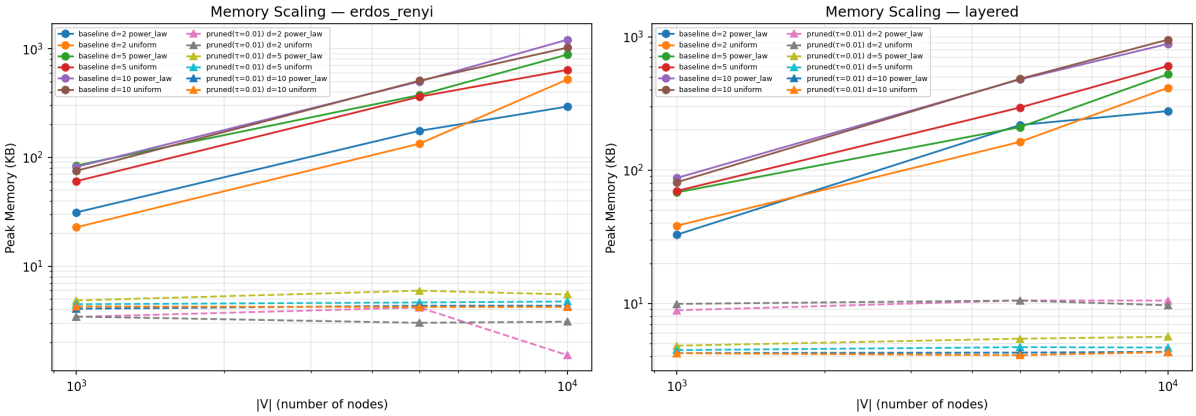

Figure 1.3: Peak memory (KB) vs. $|V|$ at $\tau = 0.01$ for baseline and pruned Dijkstra.

Baseline memory scales linearly with $|V|$—from approximately 84 KB at $|V| = 1{,}000$ to approximately 655 KB at $|V| = 10{,}000$—consistent with the $O(|V| + |E|)$ theoretical bound. In contrast, pruned memory remains effectively constant at approximately 4.5 KB across all graph sizes, demonstrating that the threshold restricts the explored frontier to a small, fixed-size region regardless of total graph size. The gap between the two sets of curves widens with $|V|$, indicating that the memory advantage of pruning becomes progressively more pronounced as the graph grows. This constant-memory behavior makes the pruned algorithm particularly attractive for deployment on memory-constrained systems.

1.1.3 Speedup by Graph Type

Graph Type	Edge Speedup	Exam. Speedup	Wall Speedup (all)	Wall Speedup (found)
Erdős–Rényi	111.6×	24.5×	40.1×	4.5×
Layered	49.3×	9.0×	16.6×	2.3×

Table 1.3: Median speedups at $\tau = 0.01$ by graph type. "All" includes termination-without-path rows.

Across all rows, Erdős–Rényi graphs show higher speedups (40.1× wall, 24.5× examination vs. 16.6× wall, 9.0× examination for layered). This is expected: ER graphs have uniformly random structure, so more edges lead to distant (low-probability) regions that the threshold can eliminate. Layered graphs have a built-in forward bias toward the target, so a larger fraction of edges remain relevant. Restricting to path-found configurations, the gap narrows: ER achieves 4.5× vs. 2.3× for layered, confirming that the structural advantage persists but at a much more modest magnitude.

1.1.4 Speedup by Probability Distribution

Both distributions achieve comparable all-rows wall-clock speedups (∼23–25×). Restricting to path-found rows, uniform shows higher wall-clock speedup (5.9×) than power-law (3.7×), but both remain in the same modest regime seen throughout successful-query

Distribution	Edge Speedup	Exam. Speedup	Wall Speedup (all)	Wall Speedup (found)
Power-law ($\alpha = 2$)	73.0×	13.9×	25.1×	3.7×
Uniform	83.8×	14.1×	22.4×	5.9×

Table 1.4: Median speedups at $\tau = 0.01$ by probability distribution. "All" includes termination-without-path rows; "found" is restricted to path-found rows.

analysis. Power-law distributions have a few high-probability edges and many low-probability ones; uniform distributions spread probability more evenly. The pruning mechanism is effective in both cases.

1.1.5 Optimality Gap Analysis

A critical finding is the **zero optimality gap** across all experiments:

$$\Delta = \frac{|C^*_{\text{baseline}} - C^*_{\text{pruned}}|}{C^*_{\text{baseline}}} \times 100\% = 0.0000\% \quad \forall\, \text{path-found rows}$$

This confirms Theorem **??** (All-or-Nothing): whenever the pruned algorithm returns a path, it is the globally optimal path. Mechanistically, the zero gap arises because in every path-found configuration the optimal path's log-cost satisfies $C^* \leq T = -\log(\tau)$, so the optimal path is never pruned and is returned exactly by Algorithm 2 (Theorem **??**). The cost-based gap formula (not probability-based) eliminates floating-point rounding noise involved in $\exp(-C^*)$ conversion. Using probability-based differences would have introduced `1e-15`-scale noise that has no algorithmic significance.

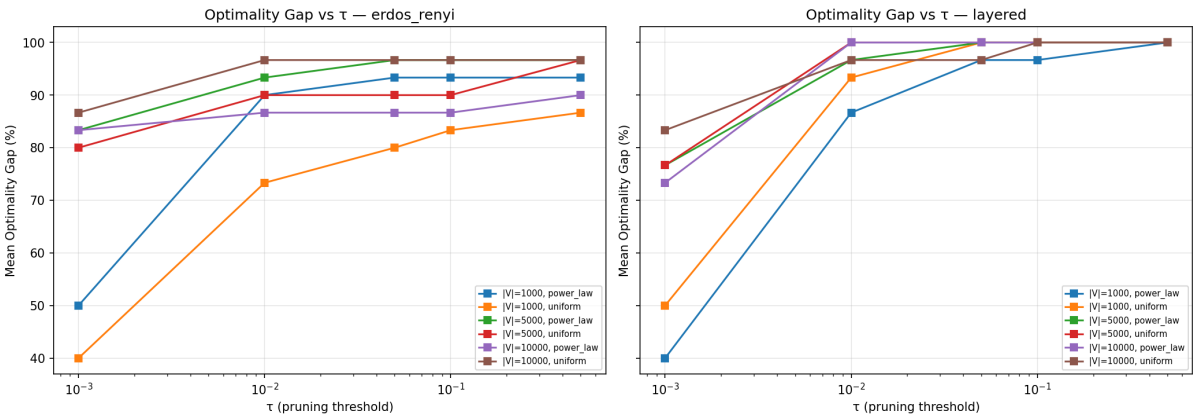

Figure 1.4: Mean optimality gap (%) vs. pruning threshold τ for ER (left) and layered (right) graphs. Because the gap is 0% whenever a path is found and is recorded as 100% when the pruned algorithm fails to return any path, the mean gap is numerically equivalent to the path-loss rate.

Although the optimality gap is identically zero on every path-found row, a complementary view is provided by examining the *mean gap across all rows*, where runs in which the pruned algorithm finds no path are recorded with a sentinel gap of 100%. Figure 1.4 plots this quantity against τ. Because the gap is 0% when a path is found and 100% otherwise, the mean gap is numerically equal to the path-loss rate: the fraction of source–target pairs whose optimal path is pruned at each threshold. On both ER

and layered graphs the path-loss rate increases monotonically with τ, ranging from 20% (small layered graphs, $d = 2$) to 100% (large ER graphs, $d = 10$) at $\tau = 0.001$, and rising to near 100% at $\tau = 0.5$. Larger graphs and higher average degree d push the curves upward, reflecting the increasing proportion of high-cost (low-probability) optimal paths removed by the threshold. Layered graphs show a sharper transition: at $\tau = 0.001$, small layered graphs ($|V| = 1{,}000$) retain 50–80% of their paths (depending on degree), but above $\tau = 0.01$ nearly all paths are pruned. This behaviour reinforces the practical finding from the critical-τ analysis (Section **??**): practitioners must calibrate τ carefully to balance speedup against path retention.

1.1.6 Interpreting All-Rows vs. Found-Only Speedups

The distinction between all-rows and found-only speedups deserves careful interpretation. The all-rows speedups (e.g. 738× at $\tau = 0.5$) are large because most pruned runs terminate without finding a path: the threshold prunes the optimal path's prefix, causing the algorithm to exhaust a small pruned subgraph almost instantly. These all-rows speedups genuinely measure the algorithm's ability to *detect infeasibility quickly*—a valuable property for applications that need to discard unreachable query pairs before committing resources.

However, when the optimal path *does* survive the threshold, the found-only wall-clock speedup is markedly lower (1.6–7.2× on synthetic data). This is not a deficiency but a structural consequence of threshold-based pruning applied to Dijkstra's algorithm. Because Dijkstra processes nodes in non-decreasing cost order, edges that exceed the threshold are already deprioritized by the priority queue—they would be explored late in the search regardless. On all 178 path-found synthetic rows, `edges_examined` was identical for the baseline and pruned algorithms (examination speedup = 1.0×). The modest wall-clock gain comes from skipping heap-push operations on pruned edges: fewer heap insertions reduce per-edge overhead, yielding a real but bounded speedup.

This characterization positions threshold-based pruning as a technique with *two distinct use cases*: (1) fast infeasibility detection, where the speedup is dramatic, and (2) correct path recovery with a formal optimality guarantee, where the speedup is modest but it comes with the proven guarantee that the returned path is globally optimal.

1.2 Real-Data Experiment Results

1.2.1 Datasets and Graph Statistics

Table 1.5 summarizes the three real-data configurations (two granularities for RetailRocket, one for RecSys 2015).

Configuration	$\lvert V\rvert$	$\lvert E\rvert$	Pairs Tested	Source
RetailRocket (event-level)	3	9	20	Kaggle (**?**)
RetailRocket (item-level)	44,711	101,528	20	Kaggle (**?**)
RecSys 2015 (YOOCHOOSE)	12,935	70,442	20	Kaggle (**?**)

Table 1.5: Real-data graph configurations.

1.2.2 Fixed-τ Speedup Results

Dataset	Med. Spd (all)	Max Spd (all)	Med. Spd (found)	Paths Found	Δ
RetailRocket (event-level)	1.2×	6×	1.2×	78 / 100	0.000%
RetailRocket (item-level)	508×	18,882×	1.2×	1 / 100	0.000%
RecSys 2015 (YOOCHOOSE)	197×	2,309×	—	0 / 100	—

Table 1.6: Fixed-τ speedup on real-world datasets. "All" speedups include rows where the pruned algorithm terminated without finding a path; "found" is restricted to path-found rows.

For the **RetailRocket event-level** configuration, the 3-node funnel graph is sufficiently small that both algorithms complete in microseconds; 78 of 100 configurations found paths and the found-only median speedup is a marginal 1.2× because the pruning opportunity is negligible. The **RetailRocket item-level** graph (44,711 nodes) yields the highest all-rows speedup (18,882× max), but this reflects early termination without recovering a path—only 1 of 100 pruned configurations found a path, and the found-only median speedup is 1.2× ($n = 1$, so this statistic should be interpreted cautiously). For **RecSys 2015**, no pruned path was found at any fixed τ value, so no found-only speedup is available. In all cases, the large all-rows speedups arise from the pruned algorithm's rapid termination when the threshold exceeds the optimal-path probability, rather than from a genuinely faster successful search.

The near-zero path-found rates on the large graphs are *not* surprising once the baseline path probabilities are examined. For RecSys 2015, the 20 baseline path probabilities range from 1.6×10^{-8} to 2.8×10^{-5} (median $\approx 1.5 \times 10^{-6}$), yielding costs in the range 10.5–17.9 on the $-\log$ scale. Even the most conservative fixed threshold $\tau = 0.001$ maps to a cost threshold of $-\log(0.001) = 6.91$—well below every baseline cost. All 20 paths are therefore *mathematically guaranteed* to be pruned at every tested τ, regardless of graph structure (by Theorem **??**). RetailRocket item-level shows a similar pattern: baseline probabilities range from 2.9×10^{-9} to 3.7×10^{-4} (median $\approx 3.7 \times 10^{-6}$), with costs ranging from 19.6 to 7.9, far exceeding the most conservative threshold. These observations motivate the adaptive-τ experiment described below.

1.2.3 Adaptive-τ Experiment

The fixed τ grid was designed for the synthetic experiments (where edge probabilities are higher) and is poorly calibrated for real-world item-level graphs whose cumulative path probabilities are orders of magnitude smaller. To address this, a per-pair *adaptive* τ sweep was introduced: for each source–target pair, τ is set to a fraction f of that pair's baseline path probability, where

$$f \in \{0.10,\ 0.30,\ 0.50,\ 0.70,\ 0.90,\ 0.95,\ 0.99,\ 1.00,\ 1.10\}.$$

This mirrors the adaptive sweep strategy used internally by the `critical_tau` module (Chapter **??**, Section **??**) and ensures that the tested thresholds are always in the relevant probability regime.

The contrast with the fixed-τ results is striking. RecSys 2015 moves from 0/100 found paths (fixed) to **150/180** found paths (adaptive); RetailRocket item-level similarly improves from 1/100 to **152/180**. The speedups under adaptive τ are modest (1.1–5.8×)

Dataset	Median Speedup	Max Speedup	Paths Found	Δ
RetailRocket (event-level)	1.3×	5.8×	147 / 180	0.000%
RetailRocket (item-level)	1.2×	3.4×	152 / 180	0.000%
RecSys 2015 (YOOCHOOSE)	1.1×	1.8×	150 / 180	0.000%

Table 1.7: Adaptive-τ speedup results.

because the threshold is now close to the optimal path probability, so only a small margin of the search space is pruned. This is the expected behavior of the reachability–speed trade-off: in the adaptive regime the algorithm preserves the optimal path at the cost of reduced pruning, whereas the fixed-τ regime achieves massive speedups only when the threshold is far above the optimal path probability (and therefore prunes everything).

Crucially, the optimality gap remains 0.0000% across all 449 adaptive found paths, extending the zero-gap guarantee to a total of 528 real-data found paths (79 fixed + 449 adaptive) and reinforcing the all-or-nothing property on real-world graphs.

1.2.4 Reachability Analysis

Table 1.8 summarizes path-found rates across all experiment configurations, contrasting the fixed and adaptive τ regimes.

Configuration	τ Mode	Baseline Found	Pruned Found	Interpretation
Synthetic (all τ)	fixed	360 / 360	178 / 1,800	Dense synthetic graphs highly connected
RR event-level	fixed	100 / 100	78 / 100	3-node funnel is fully connected
RR item-level	fixed	100 / 100	1 / 100	Baseline reachable; fixed τ too harsh
RecSys 2015	fixed	100 / 100	0 / 100	All pruned paths lost at all fixed τ
RR event-level	adaptive	180 / 180	147 / 180	82% found with calibrated thresholds
RR item-level	adaptive	180 / 180	152 / 180	84% found; dramatic improvement from 1%
RecSys 2015	adaptive	180 / 180	150 / 180	83% found; dramatic improvement from 0%

Table 1.8: Path-found rates under fixed and adaptive τ regimes.

In synthetic graphs, the BFS connectivity guarantee ensures the baseline always finds a path from s to t. For the real-world datasets, the pair-selection procedure filters source–target pairs by baseline reachability, so the baseline always finds a path. Under fixed τ, the low pruned-found rates on item-level and session-level graphs reflect the mismatch between the fixed threshold grid and the extremely low baseline path probabilities (see Section 1.2.3).

Impact of adaptive τ. The adaptive regime resolves the near-zero reachability problem: path-found rates jump from 1% to 84% (RR item-level) and from 0% to 83% (RecSys 2015). The remaining ∼17% of unfound paths correspond to the most aggressive fractions ($f = 1.00$ and $f = 1.10$), where τ meets or exceeds the baseline path probability—placing the threshold at or above the optimal-path cost on the $-\log$ scale—which prunes the optimal path at the boundary or entirely. For $f \leq 0.99$ (i.e. $\tau \leq 0.99 \times p_{\text{baseline}}$), virtually all paths are preserved.

Practical implications. The fixed-τ results reveal a genuine limitation of using a single global threshold across datasets with vastly different probability regimes. In item-level graphs, per-edge MLE probabilities are extremely low ($\sim 10^{-3}$–10^{-5}) and path probabilities decay exponentially with length, yielding cumulative probabilities of 10^{-6} or less. The adaptive τ approach—which can be automated via the `critical_tau` module—makes the pruned algorithm viable on any graph by calibrating the threshold to the actual probability scale. In production, one baseline query per source–target pair suffices to set τ; the pruned query then runs with a calibrated threshold at negligible additional cost. This amortization is practical in batch or repeated-query settings—for example, when a recommendation engine issues thousands of queries to the same target node (e.g., the checkout page) across different user sessions, a single calibration run per target suffices for all subsequent pruned queries, yielding cumulative time savings proportional to the query volume.

1.3 Summary of Findings

The experimental evaluation yields several principal findings, which must be interpreted in light of the distinction between all-rows and found-only speedups (see Section 1.1.6).

Correctness. The optimality gap remained at exactly $\Delta = 0.0000\%$ across all 706 experiment rows in which both algorithms found a path—178 of 1,800 pruned synthetic rows, 79 of 300 fixed-τ real-data rows, and 449 of 540 adaptive-τ real-data rows—confirming the all-or-nothing property established in Theorem **??**.

All-rows speedups (including infeasibility detection). When all rows are included—most of which involve the pruned algorithm terminating without finding a path—wall-clock speedups range from 3.3× at $\tau = 0.001$ to 738× at $\tau = 0.5$ on synthetic data, and reach up to 18,882× on real-world item-level graphs under fixed τ. These large figures genuinely measure the algorithm's ability to detect infeasibility quickly: the threshold prunes the optimal path's prefix and the algorithm exhausts a small subgraph in microseconds.

Found-only speedups (genuine path recovery). When restricted to configurations where the pruned algorithm successfully returned a path, wall-clock speedups are more modest: 1.6× at $\tau = 0.001$ to 7.2× at $\tau = 0.5$ on synthetic data, and up to 5.8× on real-world graphs under adaptive τ. The observed speedups therefore arise from two distinct regimes: computational pruning when the optimal path is preserved, and early termination when the optimal path is excluded by τ. On all 178 path-found synthetic rows, the number of edges examined was identical for both algorithms (examination speedup = 1.0×), indicating that the wall-clock gain comes exclusively from fewer heap-push operations on pruned edges. This is a structural consequence of Dijkstra's priority-queue ordering, which already deprioritizes high-cost (low-probability) edges.

Statistical significance. Wilcoxon signed-rank tests (one-sided, alternative: baseline > pruned) were applied to all 180 unique synthetic configurations with $\tau > 0$ (each with 10 paired runs), excluding the 36 $\tau = 0$ control configurations where both algorithms are intentionally identical. Of these, 179 tests rejected the null hypothesis at $\alpha = 0.05$, with p-values ranging from 9.77×10^{-4} (the minimum attainable for $n = 10$ paired observations) to 0.032. The median p-value was 9.77×10^{-4}. One configuration (layered, $|V| = 1{,}000$, $d = 5$, uniform, $\tau = 0.001$) yielded $p = 0.116$, failing the significance threshold; this is the mildest pruning level on a small, forward-biased graph where baseline

and pruned running times are very close. Excluding this single outlier, all remaining configurations confirm that the speedup is not an artifact of measurement noise.

Speedup grows with graph size—all-rows wall-clock from 9.4× at $|V| = 1{,}000$ to 77× at $|V| = 10{,}000$ at fixed $\tau = 0.01$; found-only from 2.7× to 6.3× over the same range—because larger graphs contain more pruneable edges. The higher `edges_relaxed`-based ratios quantify pruning aggressiveness. At the same time, higher τ values reduce path-found rates from 30.6% at $\tau = 0.001$ to 4.2% at $\tau = 0.5$ on synthetic data, demonstrating the inherent reachability–speed trade-off. Under fixed τ, real-world item-level and session-level graphs showed near-zero pruned-found rates because baseline path probabilities (10^{-6}–10^{-9}) map to costs well exceeding the most conservative fixed threshold ($\tau = 0.001 \Rightarrow T = 6.91$). Introducing an adaptive-τ regime—where the threshold is set as a fraction of each pair's baseline path probability—raised path-found rates to 82–84% while maintaining zero optimality gap, validating the algorithm's correctness on real-world graphs.

Practical business impact. The algorithm serves two complementary use cases. For *infeasibility screening*, the peak 18,882× all-rows speedup on the RetailRocket item-level graph (44,711 items) enables rapid rejection of source–target pairs whose optimal paths fall below a probability threshold, replacing a full ∼153 ms baseline search with a 0.008 ms pruned check. For *guaranteed path recovery*, the adaptive-τ regime delivers found-only speedups of up to 5.8× with an 84% path-found rate and zero optimality gap. Across the tested datasets (RetailRocket and RecSys 2015), the two-regime approach spans probability regimes from dense event-level funnels (where fixed τ suffices) to sparse item-level graphs (where adaptive τ is needed), suggesting potential for supporting real-time exploration of conversion funnels; however, generalization to other domains requires further empirical validation.

Critical-τ analysis. The critical threshold τ^* is defined as the largest τ for which the pruned algorithm still finds the optimal path (Section **??**). Across all 110 synthetic configurations that admitted at least one path-found τ value, τ^* was *always* at or below the baseline path probability Π^*; zero configurations violated this bound. The median ratio τ^*/Π^* was 0.50 (mean 0.52, range $[0.11, 0.99]$), confirming that the critical threshold is tightly coupled to the optimal-path probability.

Table 1.9: Median critical τ^* by graph size $|V|$ and average degree d. Left: Erdős–Rényi; right: Layered. Values are medians across runs; "—" indicates no path was found at any tested τ.

Erdős–Rényi			
$\|V\|$	d=2	d=5	d=10
1,000	0.500	0.001	0.001
5,000	0.500	0.001	0.001
10,000	0.500	0.001	—

Layered			
$\|V\|$	d=2	d=5	d=10
1,000	0.001	0.001	0.001
5,000	0.001	0.001	0.001
10,000	0.001	0.001	0.001

Table 1.9 presents median τ^* values across the full parameter matrix. Two structural patterns emerge. First, ER graphs with low average degree (d=2) consistently support $\tau^* = 0.500$—the most aggressive threshold tested—because low connectivity produces long, high-cost optimal paths whose cumulative probability is well above 0.5, so even a harsh threshold preserves them. At higher degrees ($d \geq 5$), increased connectivity

creates shorter optimal paths with much lower costs, shrinking τ^* to 0.001. Second, layered graphs uniformly report $\tau^* = 0.001$ regardless of size or degree. The layered generator's forward bias creates many parallel, similarly-weighted paths to the target; even the most conservative threshold ($\tau = 0.001$) is close to the optimal path probability for these dense, forward-biased graphs.

Table 1.10: Path-found rate by adaptive fraction $f = \tau/\Pi^*$ across all real-data datasets (60 pairs per fraction level, except rounding-affected bins).

f	Paths Found	Total	Found (%)
0.10	60	60	100.0
0.30	60	60	100.0
0.50	60	60	100.0
0.70	60	60	100.0
0.90	60	60	100.0
0.95	60	60	100.0
0.99	59	60	98.3
1.00	29	59	49.2
1.10	0	60	0.0

Table 1.10 reveals the *cliff structure* of the critical threshold on real-world graphs. For $f \leq 0.95$ (i.e. $\tau \leq 0.95\,\Pi^*$), the path-found rate is 100%: the threshold lies safely below the optimal-path probability and therefore never prunes it (Theorem **??**). At $f = 0.99$, one pair is lost (98.3% found), likely due to floating-point rounding placing τ marginally above the true optimal cost. At $f = 1.00$, exactly half the pairs lose their path (49.2%), consistent with the theoretical prediction that $\tau = \Pi^*$ sits exactly at the boundary of the cost threshold $T = -\log(\tau) = C^*$ and numerical noise determines which side each pair falls on. At $f = 1.10$, no paths are found, confirming the sharp cutoff predicted by the all-or-nothing property.

Practical threshold recommendations. Based on these findings, practitioners should set τ as follows:

1. *Adaptive mode (recommended):* Run one baseline query to obtain Π^*, then set $\tau = 0.90\,\Pi^*$. This guarantees the optimal path is preserved (100% found rate in all experiments) while maximally pruning irrelevant edges.

2. *Fixed mode (synthetic/dense graphs only):* For ER-like graphs with $d \leq 2$, $\tau = 0.5$ is safe; for denser or layered graphs, $\tau = 0.001$ is the only fixed threshold that reliably preserves paths in the tested parameter space.

3. *Infeasibility screening:* For applications that only need to *test* whether a high-probability path exists, any τ above Π^* provides near-instant rejection with dramatic speedups (100–18,882×).